the framework looks for all validation attributes on those properties and executes them. Any failures result in an entry being added to a `ModelStateDictionary`—a dictionary-like structure where validation errors are stored. This is made available in the `PageModel` class via `ModelState`. It has a property named `IsValid` that returns `false` if there are any entries. The recommended practice for handling form submissions on the server is to test `ModelState.IsValid`, process the form submission if there are no validation errors, and then redirect the user to another page using the post-redirect-get (PRG) pattern you have already looked at. If there are errors, you redisplay the current page with the form.

To test this out, you should first remove the `ValidationScriptsPartial` tag helper from the Country Create page, so client-side validation is disabled for the form. Then make the following changes to the `PageModel` class.

Listing 5.28 Changing the `OnPost` handler to use the PRG pattern

```
[TempData]                                                          ❶
public string CountryCode { get; set; }                             ❶
[TempData]                                                          ❶
public string CountryName { get; set; }                             ❶

public IActionResult OnPost()
{
    if (ModelState.IsValid)                                         ❷
    {
        CountryCode = Input.CountryCode;                            ❸
        CountryName = Input.CountryName;                            ❸
        return RedirectToPage("/CountryManager/Success");           ❸
    }
    return Page();                                                  ❹
}
```

❶ Add two string properties, decorated with the TempData attribute.